

Project Folder Structure.

Index.php: Login page

register.php: New user signup *dashboard.php* – Displays the user's tasks after login

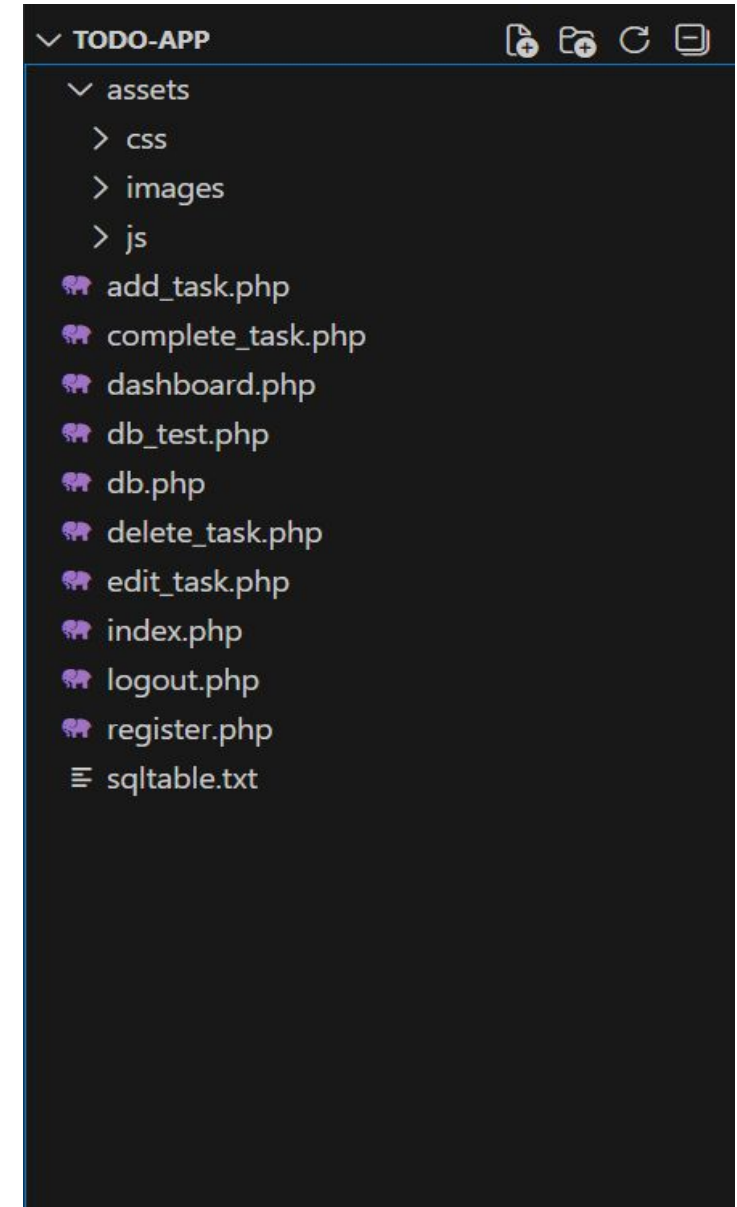
CRUD operations for tasks:

add_task.php, *edit_task.php*, *delete_task.php*,
complete_task.php

db.php: Central database + session connection (PDO)

logout.php: Ends the user session

assets/ folder: Contains CSS, JS, and images used in the app



OVERVIEW OF OUR BACKEND PAGES

Login

index.php

Shows login form

Checks user credentials in the **users** table

Starts session + redirects to dashboard

Registration

register.php

Creates new user (inserts into **users** table)

Redirects back to login

Dashboard

dashboard.php

Loads after login

Displays the user's tasks

Pulls tasks directly from the database

Task Actions (CRUD)

add_task.php, edit_task.php, delete_task.php, complete_task.php

Each updates the **tasks** table

Then returns to dashboard.php

Database Connection

db.php

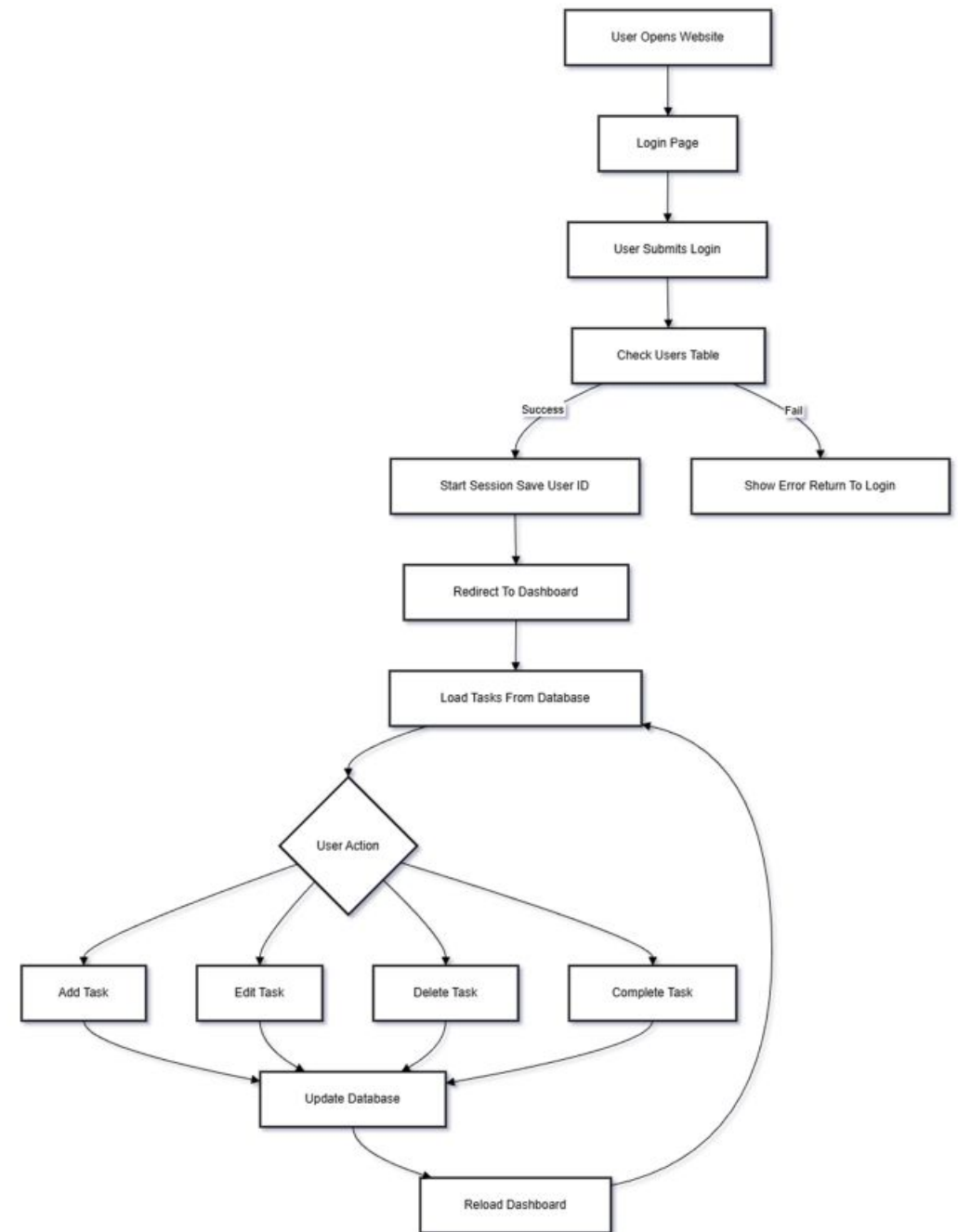
Shared by all pages

Starts session + creates PDO connection

Logout

logout.php

Ends session and returns user to login



Welcome

This is our simple to-do website made by our group 10. After you log in, you can create tasks, mark them as complete, and manage your daily work from one place.

The backend is built with PHP and MySQL.
Each user has their own tasks, which are stored in the database.

Don't have an account yet? [Create one here.](#)

Login

Email

Password

Login

New here? [Register instead.](#)

How the Backend connects to the database

- Our backend uses a centralized file called **db.php** to handle all database connections.
This file starts the user session, loads our database settings, and creates a secure PDO connection that every page reuses.
- In the next slides, we break down the **db.php** file and explain how PDO helps our backend connect safely to MySQL.



DATABASE

Database connection

The backend handles login validation, saving tasks, updating tasks, and retrieving all user data.

To keep the code organized, **db.php** was created. The db.php is a file made that starts a session and creates a secure PDO connection all pages include this file to access the same database.

Key Features of the Code

Session_start() keeps the user logged in across all pages

Database settings (host, name, username, password) define how PHP connects to MySQL

PDO is used to create a safe and modern connection to the database

ERRMODE_EXCEPTION makes debugging easier by showing helpful error messages

FETCH_ASSOC returns results as clean associative arrays (no numeric duplicates)

Centralized connection file — all pages use **db.php** instead of repeating connection code

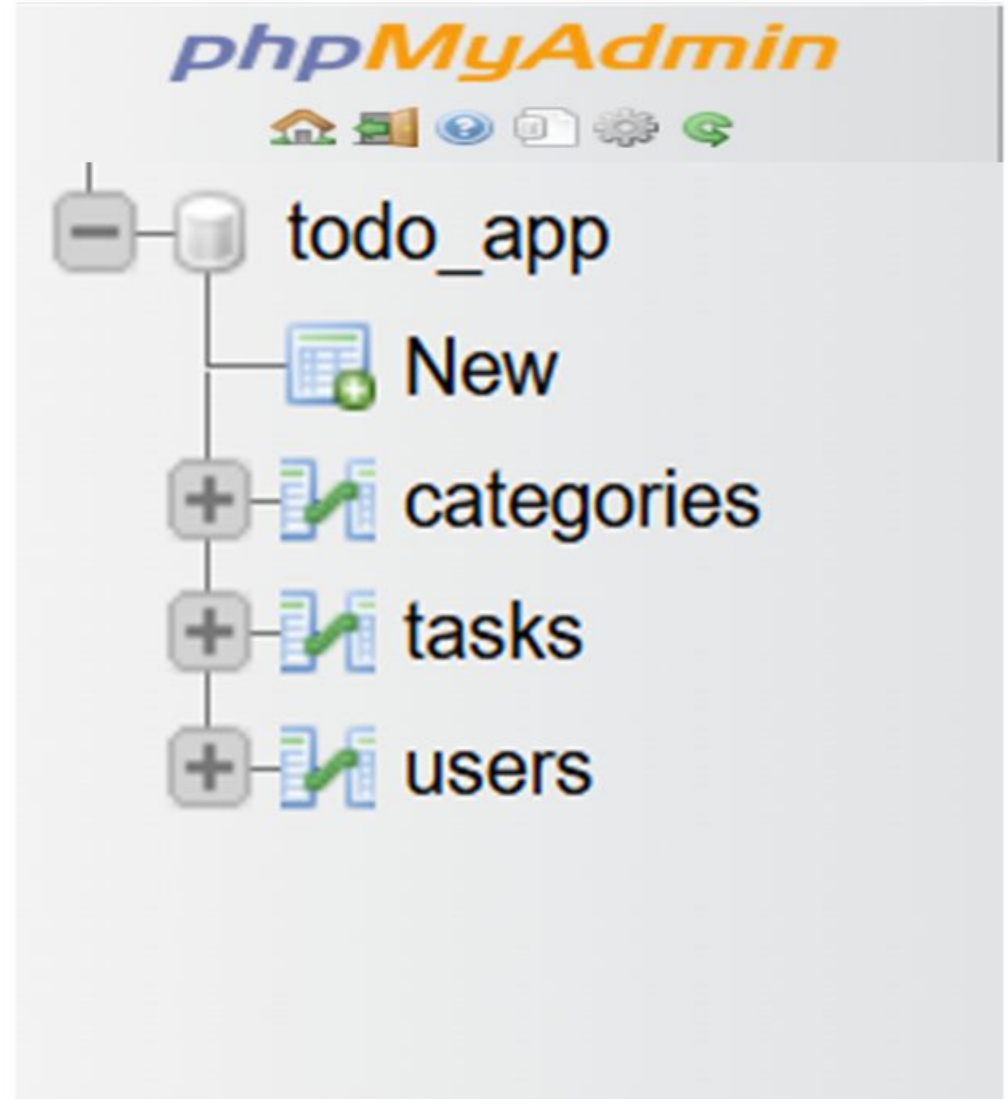
```
db.php
1  <?php
2  // db.php - database connection + session starter
3  session_start();
4
5  $DB_HOST = '127.0.0.1';
6  $DB_NAME = 'todo_app';
7  $DB_USER = 'root';
8  $DB_PASS = ''; // XAMPP default is empty - change this if you have a password
9
10 try {
11     $pdo = new PDO(
12         "mysql:host={$DB_HOST};dbname={$DB_NAME};charset=utf8mb4",
13         $DB_USER,
14         $DB_PASS,
15         [
16             PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
17             PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
18         ]
19     );
20 } catch (PDOException $e) {
21     // In development we show the error so you can fix it.
22     // In production you'd log this instead.
23     die("Database connection failed: " . $e->getMessage());
24 }
25
```

How does our site store data

Our site uses a MySQL database with three main tables: **Users**, **Categories**, and **Tasks**.

Each table handles a different part of the system and works together through foreign keys.

In the next slides, we'll break down each table and explain what it does.



Users Table

This is the users table. Every account is stored here along with its username, email, and password. The most important part is the ID, as it connects each user to their tasks. When a user logs in, we store their ID in the session, allowing us to load the appropriate tasks from the database.

Run-down of columns:

id – Primary key, unique ID for each user
username – Name chosen by the user (no duplicates)
email – Used for login (unique)
password – User's password (in real apps, this is hashed)
created_at – Timestamp showing when the user registered

What is the use of the User Table?

Stores login information

Identifies each user using their unique ID

ID is used in the tasks table (user_id) to load only the tasks belonging to that user

SQL STRUCTURE- shows main Columns.

```
USE todo_app;

-- =====
-- 1) USERS TABLE
-- Stores accounts for people who can log in
-- =====

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,    -- unique ID for each user (1,2,3...)
  username VARCHAR(100) NOT NULL UNIQUE, -- username, must be filled and not duplicated
  email VARCHAR(255) NOT NULL UNIQUE,    -- email, must be filled and unique
  password_hash VARCHAR(255) NOT NULL,   -- hashed password (from PHP password_hash)
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- when the user was created
);
```

Task Table – stores all to do items

This is the tasks table. Every Task belongs to a specific user through the ***user_id*** foreign key.

The table stores details like the title, description, due date, and completion time. We added time stamps, such as ***created-at*** and ***updated-at***, so we could track when tasks were added and modified to improve the user experience.

We made use of two foreign keys – one linking tasks to users, and another linking to categories.

If a user is deleted, all their tasks are also deleted automatically *ON DELETE CASCADE*. For categories, if one is removed, the tasks don't break- the category just becomes NULL.

SQL TABLE DEFINITION- Tables

```
-- =====
-- 3) TASKS TABLE
-- Stores all to-do items for users
-- =====
CREATE TABLE tasks (
  id INT AUTO_INCREMENT PRIMARY KEY,      -- unique ID for each task
  user_id INT NOT NULL,                   -- which user owns this task
  category_id INT DEFAULT NULL,           -- optional link to a category (can be NULL)

  title VARCHAR(255) NOT NULL,            -- short title of the task
  description TEXT,                       -- longer description (optional)
  due_at DATETIME DEFAULT NULL,           -- when the task is due (optional)
  completed_at DATETIME DEFAULT NULL,     -- when the task was completed (NULL = not done)

  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- when the task was created
  updated_at TIMESTAMP NULL DEFAULT NULL
    ON UPDATE CURRENT_TIMESTAMP,         -- auto-updated when the row is changed

  -- user_id must refer to an existing user
  -- if the user is deleted, their tasks are deleted too
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,

  -- category_id must refer to an existing category if not NULL
  -- if the category is deleted, tasks stay but category_id becomes NULL
  FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE SET NULL
);
```


CATEGORIES TABLE

This is our categories table. Each category belongs to a specific user because categories are personal. The foreign key ensures that if a user is deleted, all of their categories are also deleted. Categories help us organize tasks into things like Work, School, or Personal.

Columns

id (INT, AUTO_INCREMENT)-Unique category ID

user_id (INT)- Links category to the user who created it

name (VARCHAR(100))- Category name (e.g., "School", "Work", "Personal")

created_at (TIMESTAMP)-When the category was created

SQL STRUCTURE.

```
);  
  
-- =====  
-- 2) CATEGORIES TABLE  
-- Stores task categories for each user (e.g. School, Work, Personal)  
-- =====  
  
CREATE TABLE categories (  
    id INT AUTO_INCREMENT PRIMARY KEY,      -- unique ID for each category  
    user_id INT NOT NULL,                   -- which user this category belongs to  
    name VARCHAR(100),                      -- category name (e.g. "School")  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- when the category was created  
  
    -- user_id must refer to an existing user  
    -- if the user is deleted, their categories are deleted too  
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

What works well?

- User authentication (register, login, logout) works smoothly
- Tasks are linked to individual users using sessions
- Full CRUD functionality for tasks (add, edit, delete, complete)
- Centralized database connection using db.php keeps code organized
- Use of PDO improves security and prevents SQL injection

What can be improved?

- Add task categories or priority levels
- Improve UI with better styling and animations
- Add password reset and email verification features
- Implement search and task filtering options